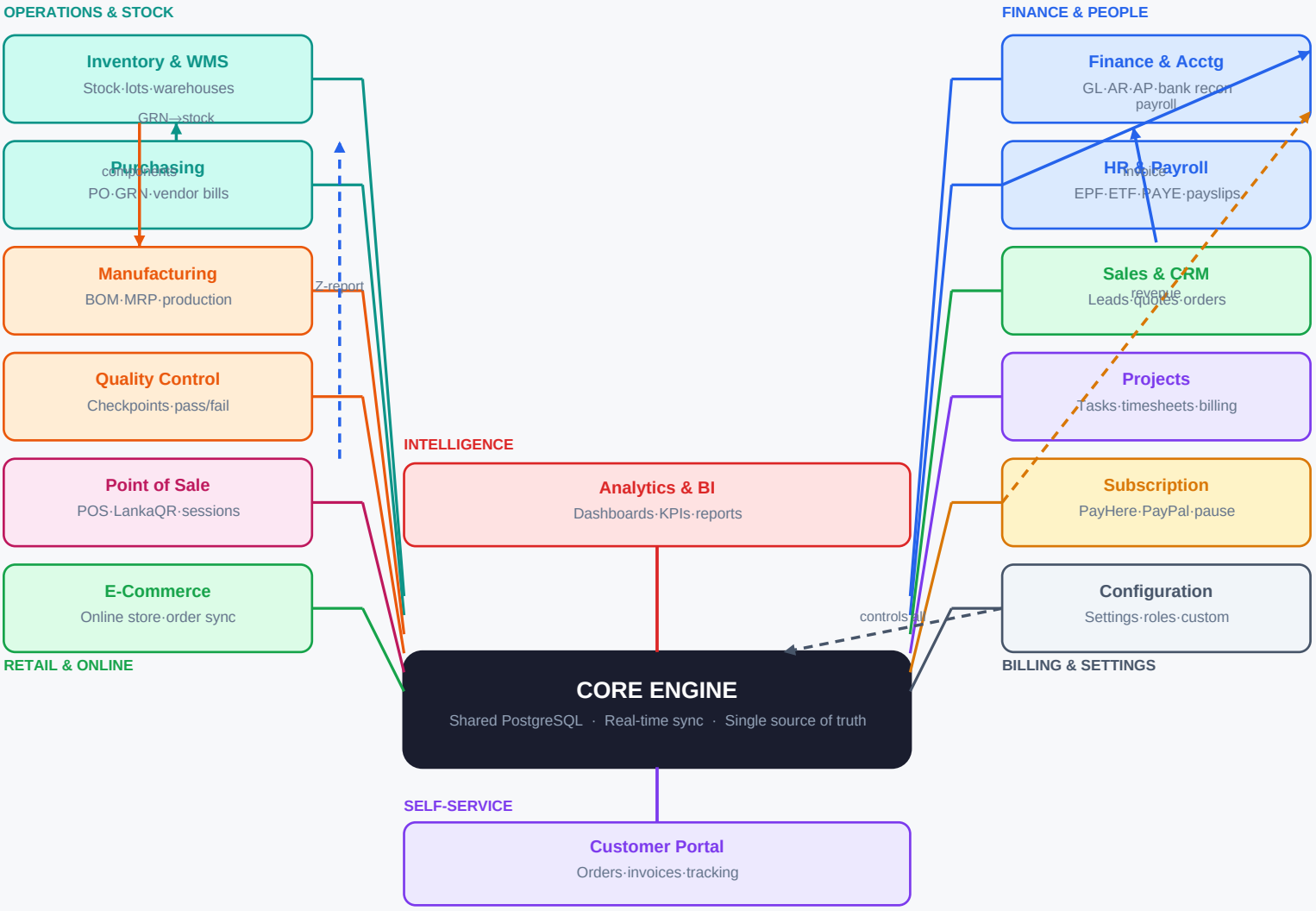
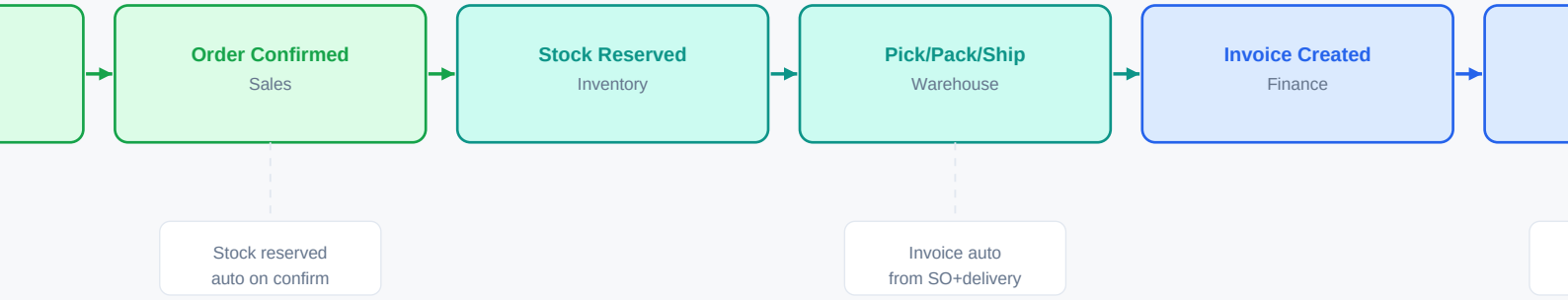


LEGEND

- Inventory/Purchasing/QC
- Manufacturing/MRP
- Point of Sale (Retail)
- Sales/E-Commerce
- Finance/HR & Payroll
- Projects/Portal
- Subscription/Billing
- Configuration
- Direct DB link
- Auto-triggered link

How the Core Engine works:

Every module writes to and reads from ONE shared PostgreSQL database. No separate systems.
When Sales confirms an order → Inventory module sees it immediately → Finance module creates the AR entry.
When HR runs payroll → Finance GL is updated in the same transaction. No exports, no imports, no manual re-entry.
The Configuration module controls which modules are ON/OFF and all behaviour rules for every tenant (company).



When Sales Order is CONFIRMED — these update automatically:

Inventory
stock_moves record created
qty_reserved += ordered_qty

Finance
account_move (AR) draft created
debit: Accounts Receivable

Analytics
sale_order KPI refreshed
revenue forecast updated

Customer Portal
order visible to customer
tracking link emailed

When Delivery (DO) is CONFIRMED — these update automatically:

Inventory
stock_moves: DONE
qty_on_hand -= delivered_qty

Finance
invoice auto-validated
AR entry = posted

Sales
SO status → Done
delivery link on SO

Analytics
on_time delivery KPI
customer fill-rate updated

When PAYMENT is received — Accounting entries (configurable per payment method):

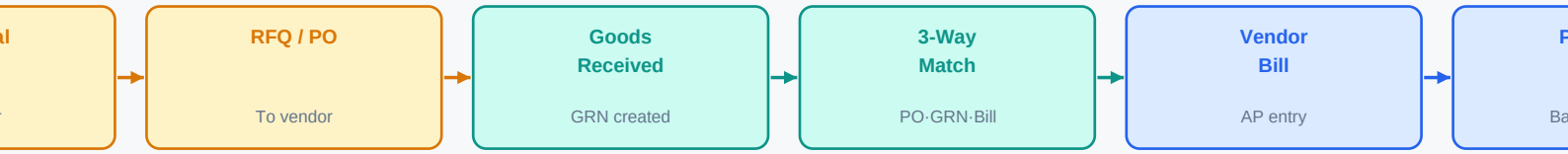
Cash payment: Dr. Cash Account (1010) Cr. Accounts Receivable (1200)
Card payment: Dr. Card Clearing (1020) Cr. Accounts Receivable (1200) [cleared when bank settles]
PayHere/LankaQR: Dr. PayHere Clearing (1025) Cr. Accounts Receivable (1200) [settled T+1 or T+2]
PayPal (USD): Dr. PayPal USD Acct (1030) Cr. Accounts Receivable (1200) + exchange gain/loss auto

Sri Lanka specifics in this flow:

VAT 18% is auto-added to every invoice line. Tax account: 2300 VAT Payable. Dr. Customer Cr. Revenue + Cr. VAT Payable.
NBT (if applicable) calculated separately. Invoice PDF generated in Sinhala/Tamil/English based on customer preference.
Payment via PayHere: webhook PAYMENT_COMPLETED fires → system auto-posts receipt → AR cleared.
Aged receivables report updates live. Customer statement auto-emails on request.

Database tables involved in this flow:

sale_order → sale_order_line → stock_picking → stock_move → account_move → account_move_line
account_payment → account_bank_statement_line (bank reconciliation)
Key fields: sale_order.state [draft|sent|sale|done|cancel] · stock_move.state [draft|confirmed|done]
account_move.state [draft|posted|cancel] · account_move.payment_state [not_paid|partial|paid|in_payment]



KEY EVENT: GRN (Goods Received Note) CONFIRMED — what happens automatically:

1. stock_move.state → DONE | product.qty_on_hand += received_qty (Inventory updated immediately)
2. purchase_order_line.qty_received updated | PO status → 'received' or 'partial'
3. account_move (vendor bill) auto-created in DRAFT state (Finance notified)
4. If Landed Costs enabled: freight/customs/duty allocated across product lines → cost price updated
5. Inventory valuation entry posted: Dr. Stock Account Cr. Stock Interim (Received not Invoiced)

3-Way Match (step 5) — system blocks payment until all three agree:

Purchase Order vs Goods Received Note vs Vendor Invoice
Tolerance rules (configurable): allow ±2% or ±LKR 500 variance before blocking.
If mismatch: system flags bill as 'Under Review'. Only Finance Manager can override.
When matched: bill.state → 'posted' | AP account credited | payment scheduled

Accounting entries — when GRN confirmed:

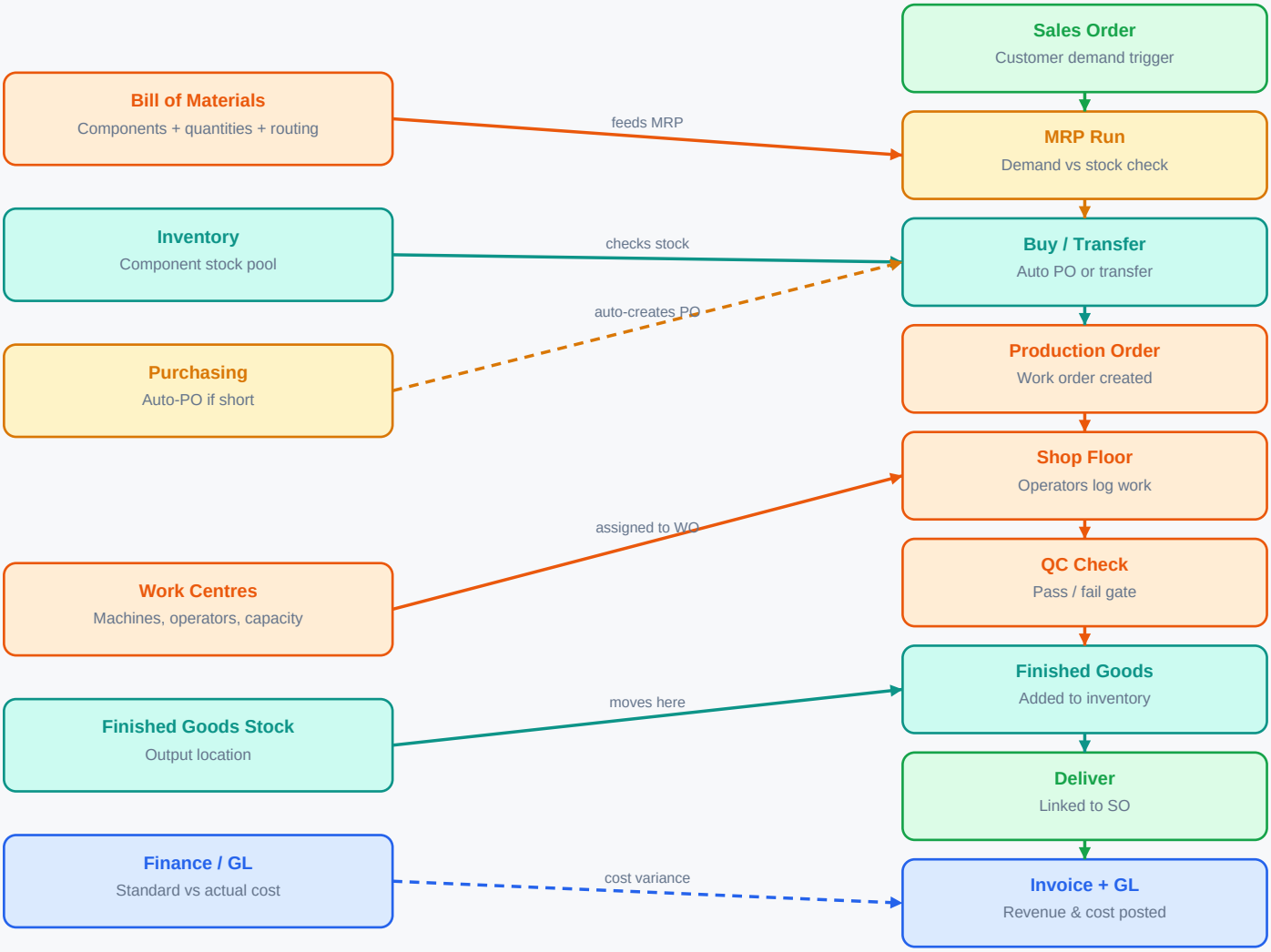
Dr. Stock Interim Received (2100) Cr. Accounts Payable (2000) [goods in, not yet invoiced]
When Vendor Bill posted (3-way match passed):
Dr. Accounts Payable (2000) Cr. Stock Interim Received (2100) [clears interim]
Dr. Purchase/Expense Account (5000) Cr. Accounts Payable (2000) [actual cost booked]
When Payment made:
Dr. Accounts Payable (2000) Cr. Bank / Cash Account (1010/1015)
Sri Lanka Import: Customs Duty → Dr. Import Duty Expense (5010) Cr. Cash/Bank

Sri Lanka Import PO — additional steps:

Landed Cost module: add freight, port handling, customs duty, insurance per shipment.
Apportion to products by: quantity / weight / value (configurable per shipment).
This increases product.standard_price automatically → COGS reflects true import cost.
Tables: stock_landed_cost → stock_landed_cost_line → account_move

Database tables in this flow:

purchase_order → purchase_order_line → stock_picking (type=incoming) → stock_move
account_move (vendor bill) → account_payment → account_bank_statement_line
Key states: purchase_order.state [draft|sent|purchase|done|cancel]
stock_picking.state [draft|confirmed|assigned|done|cancel] · account_move.state [draft|posted]

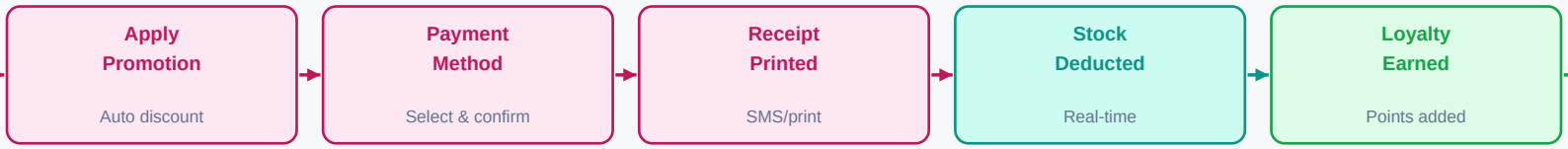


KEY EVENTS — auto-triggers in Manufacturing:

Production Order CONFIRMED: component stock_moves reserved | work centre capacity blocked
Work Order STARTED (shop floor): actual start_time recorded | operator linked | status → In Progress
Work Order DONE: actual duration vs planned → variance logged | component consumption posted
Production DONE: mrp_production.state → done | finished qty → inventory | BOM cost posted to GL
QC FAIL: production order → Rework or Scrap route | scrap cost expensed | yield % updated

Accounting entries in Manufacturing:

Component consumption: Dr. WIP Account (5100) Cr. Raw Material Stock (1300)
Labour cost: Dr. WIP Account (5100) Cr. Payroll Accrual (2200)
Overhead: Dr. WIP Account (5100) Cr. Overhead Absorbed (5200)
Finished goods: Dr. Finished Goods (1400) Cr. WIP Account (5100)
COGS on delivery: Dr. COGS (5000) Cr. Finished Goods (1400)
Variance: Dr/Cr Variance Account (5300) — standard vs actual difference



PAYMENT METHODS — Accounting Entries (each method has its own GL account, configurable):

CASH: Dr. Cash Account (1010) Cr. POS Sales Revenue (4000) + Cr. VAT Payable (2300)
CARD: Dr. Card Clearing Account (1020) Cr. POS Sales Revenue (4000) + Cr. VAT Payable (2300)
LankaQR: Dr. LankaQR Clearing (1022) Cr. POS Sales Revenue (4000) + Cr. VAT Payable (2300)
PayHere: Dr. PayHere Clearing (1025) Cr. POS Sales Revenue (4000) + Cr. VAT Payable (2300)
Store Credit: Dr. Customer Advance (2050) Cr. POS Sales Revenue (4000) + Cr. VAT Payable (2300)
SPLIT: Each portion → its own account above (system splits automatically by method chosen)

When POS SALE is PAID — what updates across the system automatically:

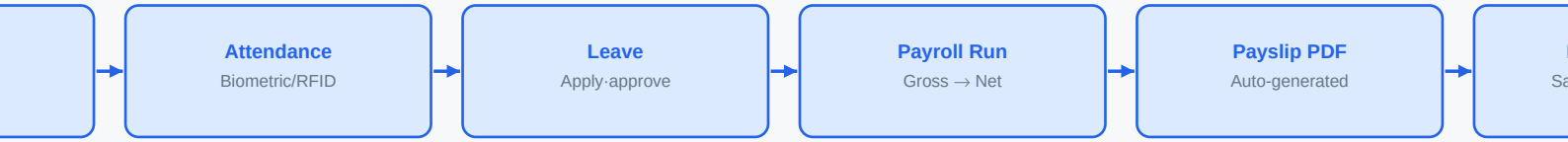
Inventory: stock_move created (state=DONE) | product.qty_on_hand -= qty_sold (immediate)
Loyalty: loyalty_points += (sale_total / points_per_LKR) | customer record updated
Analytics: pos_order posted | hourly/daily sales KPI refreshed | best-seller rank updated
Finance: pos_order_line linked to account_move_line (collected at session close)
Customer: last_purchase date updated | purchase_count++ | customer_rank recalculated
Config check: tax_rate from POS config | loyalty_rule from active promotion | payment method GL from config

POS SESSION CLOSE — Z-Report & GL posting:

System reconciles: expected_cash = opening_float + cash_sales vs counted_cash (cashier counts)
Difference → Cash Difference Account (Over/Short) posted automatically.
GL entries posted per payment method (one batch journal entry per session):
Dr. Cash (1010) Cr. POS Revenue (4000) for cash portion
Dr. Card Clearing (1020) Cr. POS Revenue (4000) for card portion
Dr. LankaQR Clearing(1022)Cr. POS Revenue (4000) for QR portion
Cr. VAT Payable (2300) for total VAT across all orders in session

OFFLINE MODE — how it works:

POS browser caches product catalogue, price lists, tax rules, customer list using IndexedDB.
Transactions stored locally (pos_order table in localStorage) with status='offline'.
When connection restored: sync worker sends all offline orders to server via POST /api/pos/orders/sync
Server processes each order, creates stock_moves, updates inventory, returns confirmation.
stock.qty_on_hand updated on server. POS display refreshed. Conflict resolution: server wins.



PAYROLL RUN — calculation sequence (Sri Lanka):

1. Get all active employees with payroll_period = current month
2. Fetch attendance records → calculate worked_days, OT_hours
3. Gross = basic_salary + OT + allowances - unpaid_leaves
4. EPF_employee = gross × 0.08 | EPF_employer = gross × 0.12
5. ETF_employer = gross × 0.03
6. PAYE = calculate_paye(gross - EPF_employee, IRD_slabs) [annual slabs divided by 12]
7. Net = gross - EPF_employee - PAYE - other_deductions

PAYROLL ACCOUNTING ENTRIES (posted automatically when payroll approved):

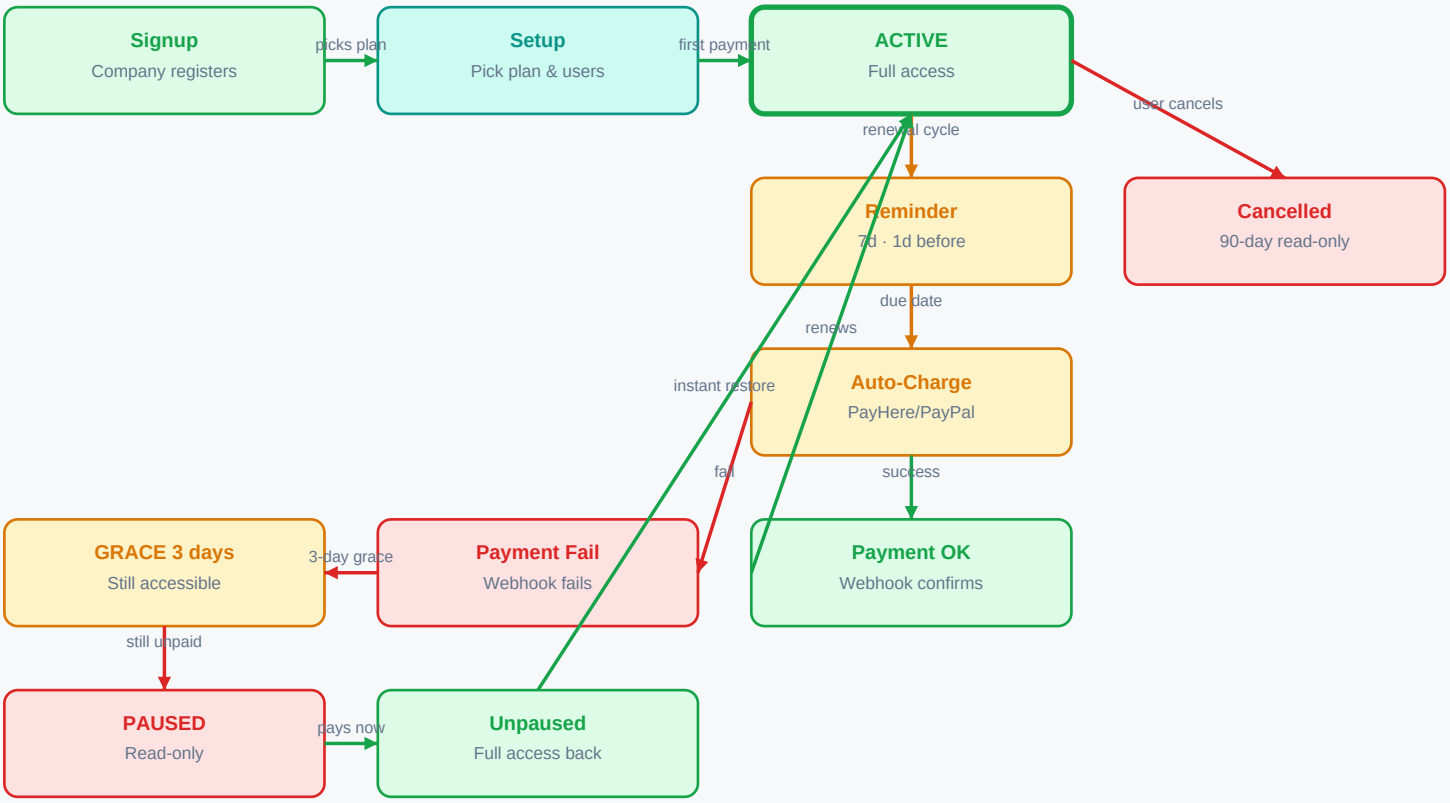
Dr. Salary Expense (6100) Cr. Salaries Payable (2300) [gross salary accrual]
Dr. EPF Contribution Exp. (6110) Cr. EPF Payable (2310) [employer 12%]
Dr. ETF Contribution Exp. (6120) Cr. ETF Payable (2320) [employer 3%]
Dr. Salaries Payable (2300) Cr. EPF Payable (2310) [employee 8% deducted]
Dr. Salaries Payable (2300) Cr. PAYE Tax Payable (2330) [PAYE deducted]
Dr. Salaries Payable (2300) Cr. Bank Account (1010) [net salary paid]
Dr. EPF Payable (2310) Cr. Bank (1010) [total EPF remitted to dept of labour]

PAYE Tax Slabs — Sri Lanka IRD (2024) — configurable in Config module:

Annual income up to LKR 1,200,000 → 0% (exempt)
LKR 1,200,001 to LKR 1,700,000 → 6% on excess
LKR 1,700,001 to LKR 2,500,000 → 12% on excess
LKR 2,500,001 to LKR 3,500,000 → 18% on excess
LKR 3,500,001 to LKR 5,000,000 → 24% on excess
Above LKR 5,000,000 → 30% on excess
System divides annual slab by 12 → monthly PAYE deduction. Rates editable in Config > HR > Tax Slabs.

Bank Salary Upload — format per bank (configurable in Config > HR > Bank Upload Format):

Sampath Bank: CSV with columns: acc_no, name, amount, reference, branch_code
BOC: Fixed-width text format with header record + detail records
HNB: CSV with columns: account, name, nic, amount
Commercial: MT103 SWIFT format (for larger corporates)
System generates the correct file automatically. Finance Manager downloads and uploads to bank portal.
Tables: hr_payslip hr_payslip_line account_move account_move_line hr_bank_upload



PAUSED state — access matrix:

Login: YES (authentication works)
View records: YES (all data readable)
Download PDF: YES (invoices, reports)
Create/Edit: NO (403 Forbidden from API middleware)
POS sales: NO (pos_config.status check blocks session open)
Middleware: if tenant.subscription_status != 'active' and request.method != 'GET': return 403

Pricing calculation:

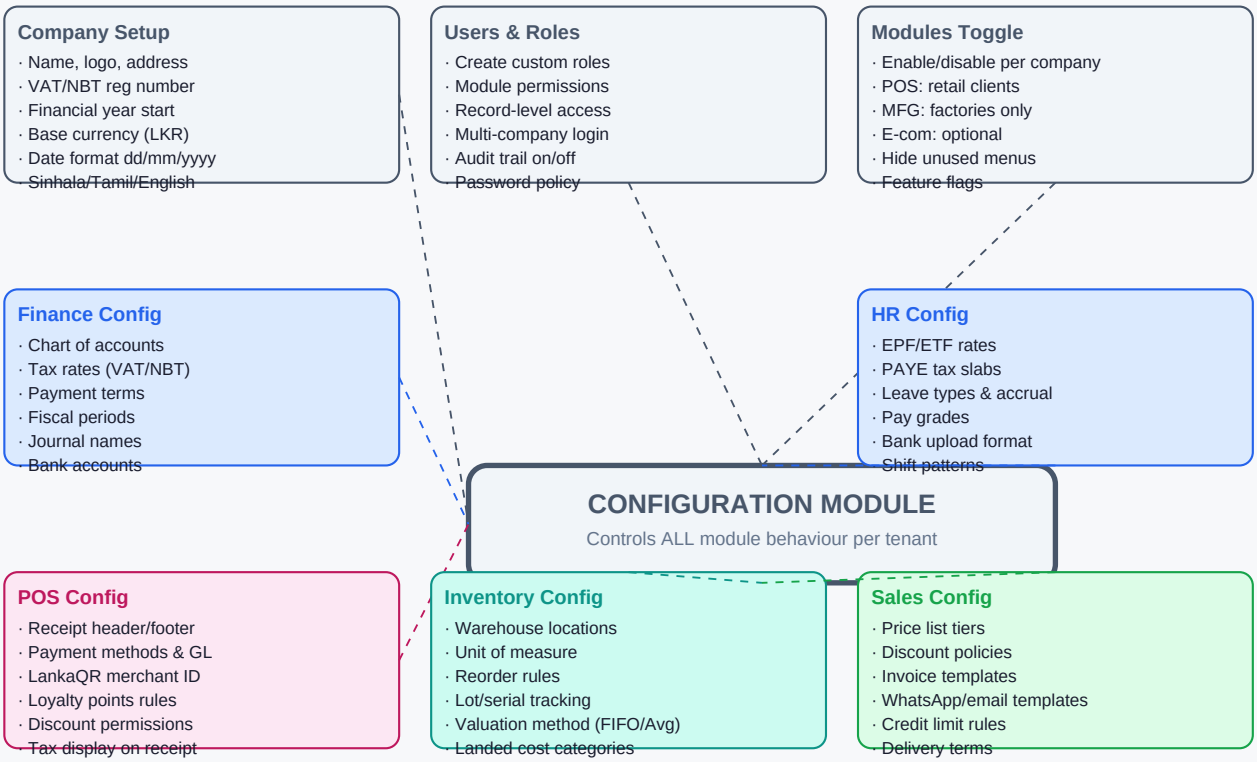
LKR 2,000 = base (1st user)
LKR 1,000 × (user_count - 1) = extra users
Total = 2000 + (users-1) × 1000
5 users = LKR 6,000/month
10 users = LKR 11,000/month
20 users = LKR 21,000/month

PayHere Integration (primary — LKR payments):

Recurring: POST https://www.payhere.lk/pay/checkout with subscription_id, amount, recurrence=1 Month
Webhook: POST /webhook/payhere → verify md5(merchant_id+order_id+amount+currency+md5_secret).upper()
status_code=2 (SUCCESS): update subscription.status='active', subscription.paid_until=+30days
status_code=0 (FAILED): update subscription.status='grace', send SMS+email reminder
PayPal Integration: POST /v1/billing/subscriptions | Webhook: BILLING.SUBSCRIPTION.RENEWED
Tables: res_tenant subscription_plan subscription_billing payment_transaction payment_webhook_log

Reminder schedule (background Celery task — runs daily at 09:00 LK time):

7 days before due: send email + SMS 'Your NexaERP subscription renews on {date}. Amount: LKR {amt}'
1 day before due: send email + WhatsApp 'Reminder: subscription renews tomorrow'
Due date: attempt auto-charge (PayHere recurring / PayPal subscription charge)
Day 1 of grace: send email + SMS 'Payment failed. You have 3 days to update payment method'
Day 3 of grace: set subscription_status='paused'. Send email 'ERP paused. Pay to restore.'
Any payment received: immediately set status='active'. All features restored within 30 seconds.



Full list of per-company customisations:

Document templates: invoice, PO, payslip, receipt, quotation — HTML/Jinja2 templates editable per company

Custom fields: add extra fields to any model (products, customers, employees) via Config > Custom Fields

Approval workflows: define who approves PO above LKR X, who approves leave, who approves invoices

Email/SMS templates: customise every notification with company name, logo, variables like {customer_name}

Payment method accounts: each POS payment method maps to a GL account (editable in Config > POS > Accounts)

Tax configuration: VAT 18%, NBT, custom taxes — each with debit/credit accounts and applicability rules

Loyalty programme: points per LKR, tier thresholds, expiry days, redemption rules — all per company

How Configuration flows into each module at runtime:

Every API request carries: {tenant_id, user_id, role_permissions}

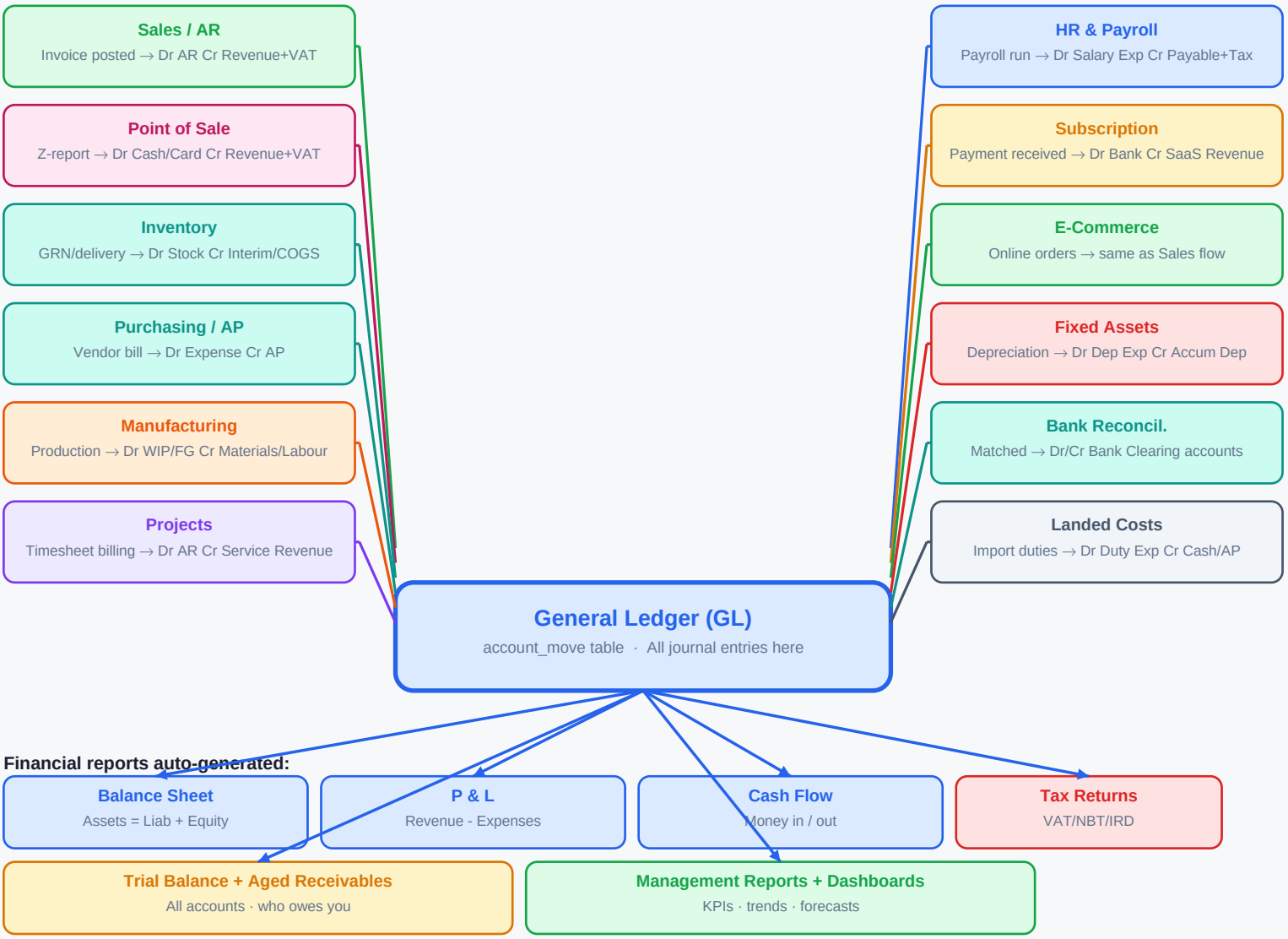
Module reads config: SELECT * FROM ir_config_parameter WHERE key LIKE 'pos.%' AND company_id = tenant_id

Tax engine reads: account_tax WHERE company_id = tenant_id AND active = true

Payment method: pos_payment_method JOIN account_journal ON journal_id WHERE company_id = tenant_id

EPF rate: hr_config_parameter WHERE key = 'epf_employee_rate' AND company_id = tenant_id

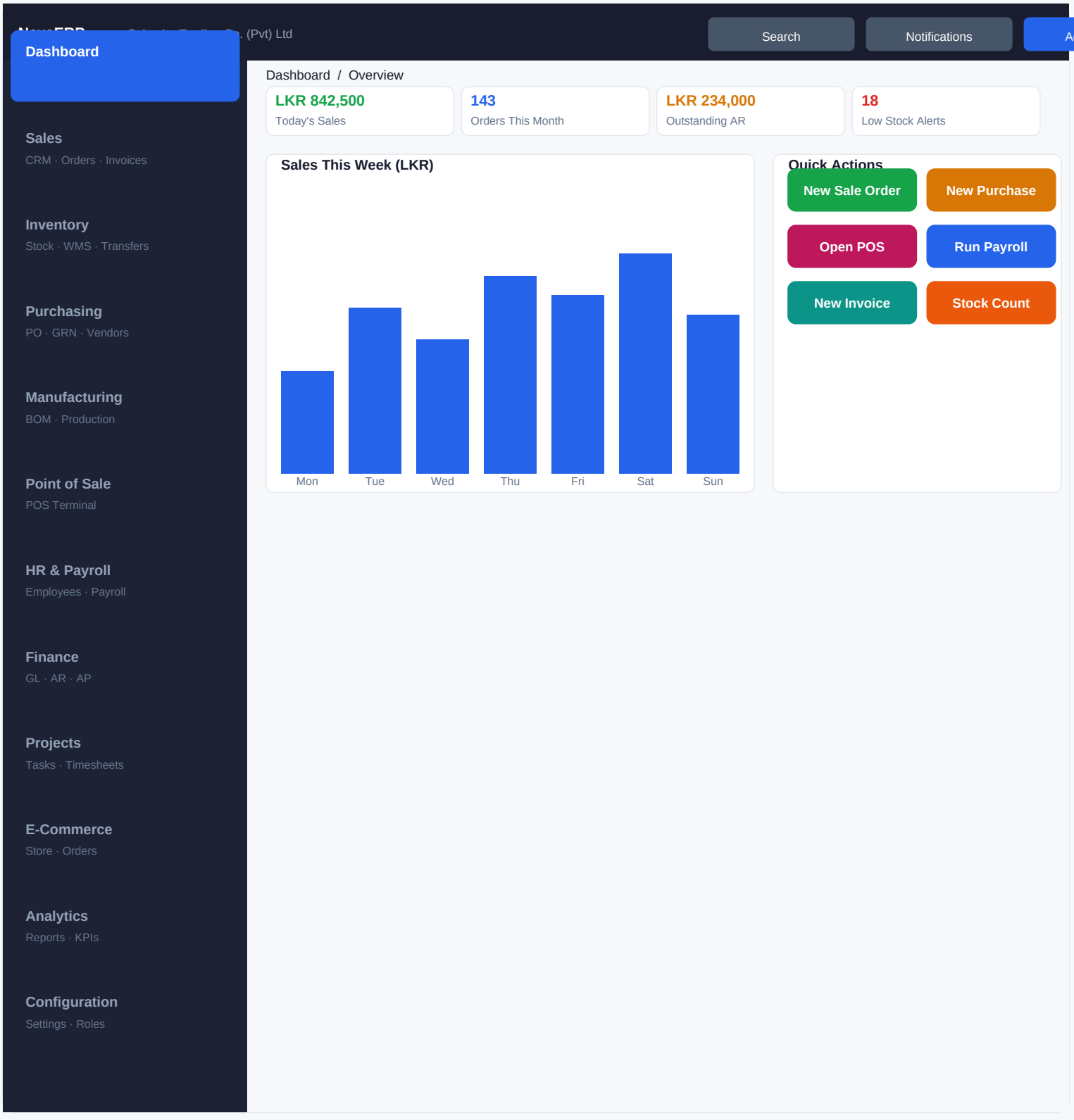
All configs cached in Redis with key: config:{tenant_id}:{module} — TTL 5 minutes for performance



Financial reports auto-generated:

Sri Lanka Tax — Auto-calculated and reported:

VAT 18%: Every sales invoice → VAT Payable (2300) credited. Monthly VAT return auto-generated.
Input VAT (purchases) → VAT Receivable (1500) debited. Net VAT = Output - Input → to pay IRD.
NBT: If applicable, separate NBT Payable account. Included in tax return.
EPF: Dr EPF Payable (2310) monthly. EPF Return (Form C) auto-generated with employee details.
ETF: Dr ETF Payable (2320) monthly. ETF Return auto-generated.
PAYE: Dr PAYE Payable (2330) monthly. PAYE Return (T10) auto-generated per employee.
All tax accounts configurable in Config > Finance > Tax Accounts. Account numbers editable per company.



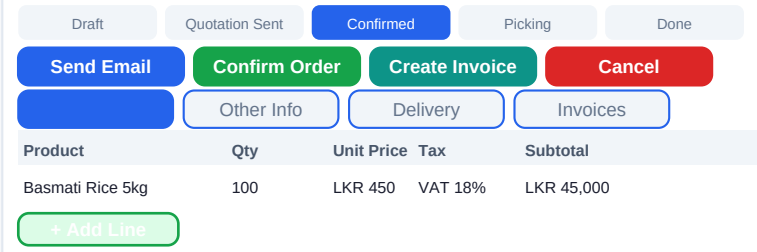
App Layout — Structure:

TOPBAR (height 30px): Logo | Company Name | Search | Notifications | User Menu (logout/profile/switch company)
SIDEBAR (width 130px): Module navigation list. Active module highlighted. Sub-items shown on hover.
CONTENT AREA (flex-1): Breadcrumb at top. Page title + action buttons. Main content below.
RESPONSIVE: Sidebar collapses to icon-only on screens < 1024px. Mobile: bottom tab bar (5 main modules).
TECH: React + Next.js. Sidebar = <Sidebar> component. Content = <Outlet> in React Router. State in Zustand.

Sales Module — Screen Map

Sales Orders (List Screen)

Sales Order SO-001 (Detail Screen)



Sales Module — Button Specifications:

+ New Order button (top-right of list): Opens blank SO form in same page (slide-in drawer or new route /sales/new)

Confirm Order button (detail): PATCH /api/sales/{id}/confirm → state=sale → stock.reserved++
Validation: at least 1 line, customer set, delivery address set

Send Email button: Opens email composer with PDF quotation attached → POST /api/mail/send

Create Invoice button (on confirmed SO): POST /api/invoices/from-sale/{id} → creates account_move in draft
Only visible when: state=sale AND delivery_status=done (or partial)

Cancel button: Confirmation modal: 'Cancel this order? Stock will be unreserved.'

Sales Module — Tab Structure (Detail Screen):

Tab 1: ORDER LINES — product table with Add/Remove/Edit inline. Fields: product, description, qty, unit, price, tax, discount, subtotal
Bottom: Subtotal / Tax / Total display. Currency selector if multi-currency.

Tab 2: OTHER INFO — customer ref, payment terms, salesperson, incoterms, fiscal position, company, analytic account

Tab 3: DELIVERY — linked stock.picking records. Shows: DO ref, scheduled date, status, tracking. Button: View Delivery

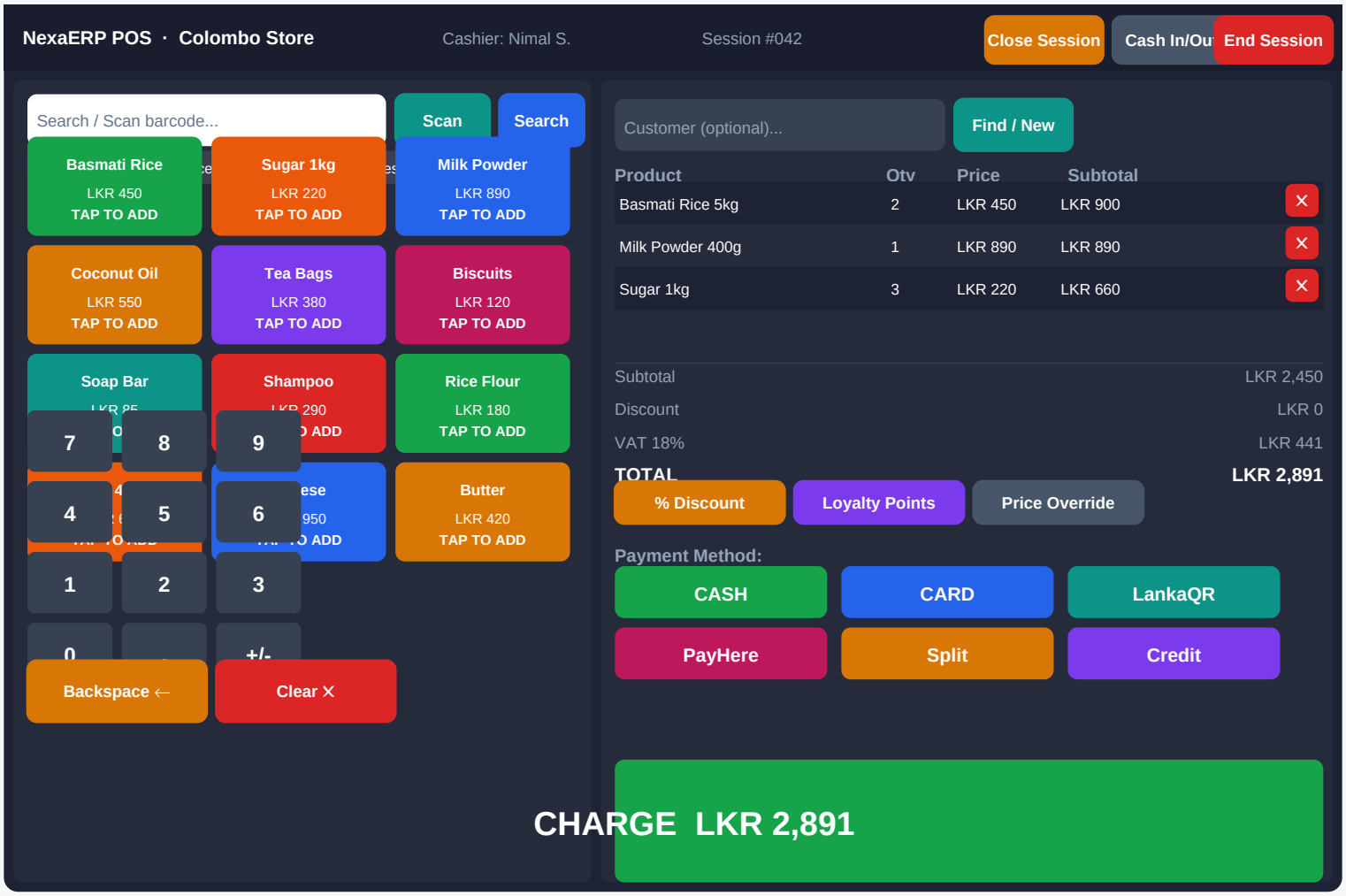
Tab 4: INVOICES — linked account.move records. Shows: invoice ref, date, amount, state. Buttons: View Invoice, Create Credit Note

Smart buttons (top of form, icon+count): Deliveries (n), Invoices (n), Emails (n) — click to see linked records

Chatter (bottom): activity log, messages, followers. Post message / Schedule Activity buttons always visible.

Sales Module — API Endpoints needed:

GET	/api/sales	List with filters: state, customer_id, date_from, date_to, page, limit
GET	/api/sales/{id}	Detail with lines, deliveries, invoices, payments
POST	/api/sales	Create new SO body: {customer_id, lines[], payment_terms, ...}
PATCH	/api/sales/{id}	Update header fields
PATCH	/api/sales/{id}/confirm	Confirm order → triggers stock reservation
PATCH	/api/sales/{id}/cancel	Cancel → unreserve stock
POST	/api/sales/{id}/send-email	Send quotation PDF
POST	/api/invoices/from-sale/{id}	Create invoice from SO



POS Screen — Button Behaviours:

Product tile TAP: POST to cart state (client-side). If qty already in cart → increment. Stock check: GET /api/pos/stock/{product_id}

Barcode scan (USB/camera): scandit/zxing decodes barcode → lookup product → add to cart

CASH payment flow: Open numpad modal for amount tendered → calculate change → show change amount prominently

CARD payment: Open card terminal integration dialog (or manual amount entry if using standalone terminal)

LankaQR: POST /api/pos/qr/generate → display QR code → poll GET /api/pos/qr/{ref}/status every 2s

PayHere: Open PayHere iframe/redirect → webhook confirms → receipt shown

CHARGE button: POST /api/pos/orders body: {session_id, lines[], payment_method, customer_id, discount}

Response: {order_id, receipt_url} → print receipt → clear cart → ready for next

POS Screen — Configuration points (all in Config > POS):

Payment methods: which are enabled, GL account per method, settlement account, clearing account

Receipt: template (HTML/Jinja2), printer type (ESC/POS, PDF), logo, footer text, WhatsApp/SMS on/off

Tax: VAT rate (default 18%), NBT, tax-inclusive or exclusive display

Loyalty: points per LKR, redemption rules, active programme on/off

Offline: cache duration, sync retry interval, conflict resolution policy

Discount: max % without PIN, roles that can override price, discount approval workflow

Session: require opening float count, require closing count, auto-close time

Buttons visible: configurable — hide 'Credit' payment if company doesn't use it

Inventory Module — Screen Map

Products List (tabs):

Tab: Products | Tab: Product Variants | Tab: Price Lists
Buttons: + New Product | Import | Export | Actions ■
Filters: by category, in-stock only, low-stock, archived
Columns: Image · Name · Ref · Category · Sales Price · Cost · On Hand · Forecast
Row click → Product Detail with tabs:
General Info | Sales | Purchase | Inventory | Accounting | Variants

Inventory Operations — main menu items:

Receipts → Incoming shipments from vendors (GRN)
Delivery Orders → Outgoing to customers (from sales)
Internal Moves → Warehouse to warehouse transfers
Returns → Customer returns / vendor returns
Scrap → Write-off damaged/expired stock
Physical Inventory → Cycle count / annual stock take

GRN Screen (Goods Received Note) — Detail

Draft | Ready | **In Progress** | Done | Cancelled

Validate GRN | **Return** | **Scrap** | **Print**

Source PO: PO-2025-0042 Vendor: Lanka Imports Pvt Ltd
Sched. Date: 15/06/2025 Location: WH/Stock

Product	Expected	Done	Lot/Serial	Location
Basmati Rice 5kg	100 kg	98 kg	LOT-2025-06	WH/Main
[illegible]	200 pcs	200 pcs	—	WH/Main

+ Add Line | Fill from PO

VALIDATE GRN button — complete sequence of auto-triggers:

1. VALIDATION: Check all lines have 'Done' qty. If under-received: popup 'Create backorder? (Yes/No)'
2. STOCK MOVES: stock_move.state → DONE for each line
3. INVENTORY: product.qty_on_hand += done_qty for each product at destination location
4. PO UPDATE: purchase_order_line.qty_received += done_qty | PO status recalculated
5. VENDOR BILL: account_move (vendor bill) auto-created in DRAFT if 'Create Bill on Receipt' config ON
6. VALUATION: If FIFO/AVCO: inventory valuation entry posted to GL
   Dr. Stock Account (1300) Cr. Stock Received Not Invoiced (2100)
7. LANDED COST: If landed cost exists for this PO: allocate cost across lines by method
8. NOTIFICATIONS: Send email/SMS to purchasing manager: 'GRN {ref} validated. {n} lines received.'
9. REORDER CHECK: For each product received, check if demand requires more — if so, suggest PO

API: POST /api/stock/pickings/{id}/validate body: {lines: [{id, qty_done, lot_id}]}

Stock Adjustment Screen (Physical Inventory):

Path: Inventory > Operations > Physical Inventory
Buttons: + New Adjustment | Start Count | Apply All | Discard
Process: Create adjustment → assign counters → they enter counted qty → supervisor reviews → Apply
APPLY button: for each line where counted != system_qty:
stock_move created (type=inventory_adjustment) | qty_on_hand updated
GL entry: Dr/Cr Inventory Adjustment Account (6500) vs Stock Account (1300)
Reason required for variance > configurable threshold (e.g. > LKR 5,000 or > 5%)

Inventory Module API Endpoints:

GET /api/inventory/products Product list with qty_on_hand, forecasted_qty
GET /api/inventory/products/{id}/moves All stock moves for a product (traceability)
GET /api/inventory/pickings List GRNs/DOs/Internals with filters
GET /api/inventory/pickings/{id} Detail with lines
POST /api/inventory/pickings/{id}/validate Validate GRN/DO → triggers all above
POST /api/inventory/adjustments Create stock adjustment
POST /api/inventory/adjustments/{id}/apply Apply adjustment → GL entry
GET /api/inventory/locations Warehouse locations tree

HR Module — Navigation Tabs

Employees | Leaves | Attendance | Payroll | Recruitment | Departments | Config

Pavroll Screen — List View

Period: June 2025 ■ Filter by Dept ■ + New Payroll Run | Run All (Batch) | Export CSV

Employee	Department	Gross	EPF Emp	EPF Emp+r	ETF	PAYE	Net	Status
Nimal Silva	Sales	LKR 85,000	6,800	10,200	2,550	3,200	74,800	Draft
Kamali Perera	Finance	LKR 120,000	9,600	14,400	3,600	8,500	102,100	Draft
Ashan Fernando	Warehouse	LKR 55,000	4,400	6,600	1,650	0	50,600	Confirmed

Payslip Detail Screen — Buttons & Fields

Compute Sheet | Confirm | Register Payment | Refund | Print Payslip

Employee:	Nimal Silva	Period:	June 2025
Department:	Sales	Job Position:	Sales Executive
Contract:	Permanent	Bank Account:	7700 1234 5678

Name	Category	Qty	Rate	Amount
Basic Salary	BASIC	1	LKR 85,000	LKR 85,000
EPF Employee Contribution	DEDUCTION	1	8.00%	LKR -6,800
PAYE Tax	DEDUCTION	1	IRD Slab	LKR -3,200

Payroll Module — Button Behaviours:

Compute Sheet: Runs payroll calculation for this employee. Reads: attendance, leaves, contract, config (EPF/ETF/PAYE rates). Populates all salary lines. Can run repeatedly before confirming.
Confirm button: Sets payslip.state = 'done'. Creates accounting entries (draft). Sends payslip PDF to employee email.
Register Payment: Sets payment_date. Creates account_payment. Marks payslip as paid. Moves acct entry to 'posted'.
Run All (Batch): Iterates all employees in period → runs Compute → Confirm → notifies manager to review
Bank Upload: GET /api/hr/payroll/{period}/bank-upload?bank=sampath → returns formatted CSV/TXT file
Print Payslip: GET /api/hr/payslips/{id}/pdf → server renders Jinja2 template → returns PDF

Leave Management — Screen and Buttons:

Leave List tabs: My Leaves | Team Leaves (manager view) | All Leaves (HR view) | Leave Allocation
+ New Leave button: Opens leave request form: type (Annual/Sick/Casual), from_date, to_date, reason
Validation: checks leave_balance >= requested_days for leave type
Approve button (Manager): Sets state=validate. Deducts from leave_balance. Notifies employee via email+SMS.
Refuse button: Sets state=refuse. Sends reason to employee. Balance not deducted.
Leave Calendar: Visual calendar view. Manager sees all team leaves. Color-coded by leave type.
Leave Analysis report: by employee, department, period — attendance % and leave trend charts

Attendance & Shift Screen:

Attendance list: Employee | Check-in | Check-out | Worked Hours | OT Hours
Import button: Upload biometric/RFID CSV export. System matches to employee by emp_id.
Manual Entry: HR can manually add attendance record with reason.
Shift Config (Config > HR > Shifts): define shift name, start_time, end_time, OT_start_after, OT_rate
OT calculation: If worked_hours > shift_hours: OT = worked_hours - shift_hours × OT_rate (e.g. 1.5x)
Attendance Report: Summary by employee/month. Used as input to payroll. Locked after payroll confirmed.

Finance Module — Main Tabs

Chart of Accounts Screen

Chart of Accounts (Config > Finance > Chart of Accounts):

Table: Code | Account Name | Type | Tags | Company | Actions

Account types: Asset | Liability | Equity | Income | Expense (maps to financial statements)

+ New Account button: form with Code, Name, Type, Currency, Deprecated checkbox

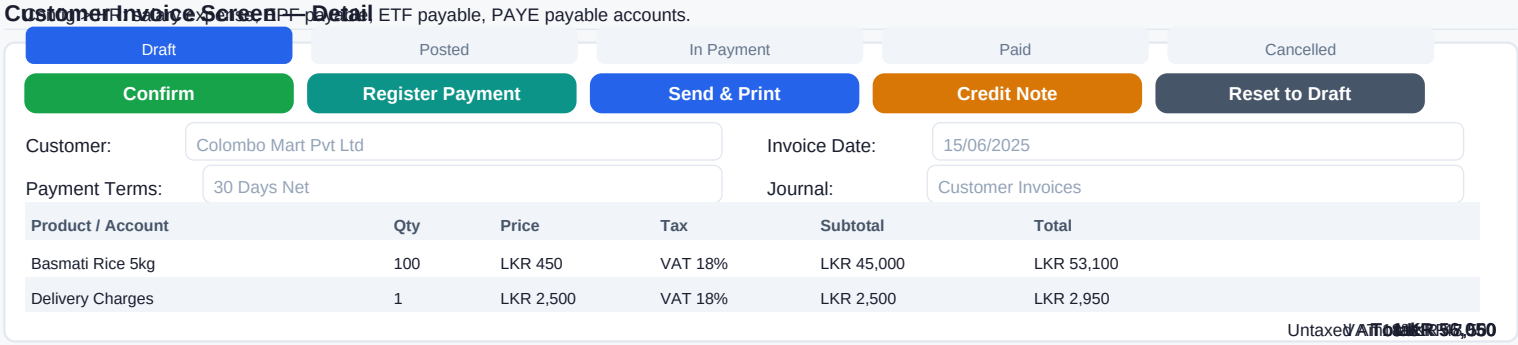
Import button: upload standard COA template (CSV). System imports and validates.

Used in: Every journal entry references an account. Config > Tax maps tax → GL account.

Config > POS > Payment Methods: each payment method has a GL account setting.

Config > Payroll > Salary Expense, EPF payable, ETF payable, PAYE payable accounts.

Customer Invoice Screen — Detail



Invoice Buttons — what they do:

Confirm button: PATCH /api/invoices/{id}/post → account_move.state=posted

Creates journal entry: Dr. Accounts Receivable (1200) Cr. Revenue (4000) + Cr. VAT Payable (2300)

Invoice number assigned: INV-2025-{sequence} | PDF generated and stored

Register Payment: Opens modal: payment_method (cash/bank/PayHere/PayPal), amount, date, memo

POST /api/payments → account_payment created → invoice.payment_state=paid

Journal entry: Dr. Bank/Cash Cr. Accounts Receivable

Send & Print: Renders PDF (WeasyPrint) → send email with PDF attached → log in chatter

Credit Note: Opens credit note wizard: reason, date → creates reversal account_move

Full reversal: exact negative of original. Partial: enter amount to credit.

Bank Reconciliation Screen:

Path: Finance > Accounting > Bank Reconciliation

Import Statement button: upload bank statement CSV/OFX/QIF → auto-creates bank statement lines

Auto-Match button: matches statement lines to account_move_lines by: amount, date ±3 days, reference

Manual Match: user selects statement line + invoice/payment → Match button

New Transaction: for bank charges, interest — creates journal entry directly

Validate button: closes reconciliation session. All matched lines marked 'reconciled'.

Sri Lanka banks: Sampath/BOC/HNB/Commercial export MT940 or CSV — import config per bank in Config > Finance > Banks

Accounting Reports (Finance > Reporting):

Balance Sheet: Assets / Liabilities / Equity as at selected date. Button: Export PDF / Excel

P&L (Income Statement): Revenue - Expenses by period. Comparison to previous period.

Cash Flow: Operating / Investing / Financing activities. Indirect method.

Trial Balance: All accounts debit/credit totals for period.

Aged Receivables: Outstanding invoices by customer, bucketed by: 0-30 / 31-60 / 61-90 / 90+ days

Aged Payables: Same for vendor bills. Both have 'Send Statement' button per customer/vendor.

VAT Return: Auto-generated from posted invoices. Output VAT - Input VAT = Net payable to IRD.

All reports: Date range selector (from/to) + Company selector + Department filter.

Configuration Module — Tab Navigation

Company | Users & Roles | Finance | HR | POS | Inventory | Sales | Email/SMS | Integrations | Modules

Config > Company Tab

Company Settings — fields:

Company Name (text) → shown on all docs
Logo (image upload) → on invoices, receipts
Address (multiline) → on invoice footer
VAT Registration No. → printed on tax invoice
Currency (select: LKR/USD/EUR) → base currency
Fiscal Year Start (month select) → Jan–Dec
Language (select: EN/SI/TA) → UI default
Date Format (select: dd/mm/yyyy etc.)

Config > Users & Roles — fields:

User list: Name | Email | Role | Last Login | Active
+ New User: Name, Email, Password (auto-generated), Role
Role editor: Name | Module | Permission (None/Read/Write/Admin)
Built-in roles: Super Admin | Manager | Accountant |
Sales Rep | Warehouse | HR Officer | Cashier | Read-Only
Custom role: start from blank, tick permissions per module
Multi-company: assign user to multiple companies

Config > Finance Tab

Finance Config — key settings:

Default Accounts (each a GL account selector):
Accounts Receivable → 1200
Accounts Payable → 2000
VAT Output (Sales) → 2300
VAT Input (Purchase)→ 1500
Bank Account → 1010
Cash Account → 1015
Tax Rates: + New Tax button
Name | Rate % | Tax Type | Invoice Acct | Refund Acct

Config > POS Tab — key settings:

Payment Methods: list of active methods
+ Add Method button: Name | Type (cash/card/qr/custom)
Journal (GL account to debit on payment)
Outstanding Acct (clearing for card/QR settlement)
e.g. LankaQR: Journal=LankaQR Clearing (1022)
Receipt Settings: Template selector | Logo on/off
Print mode: ESC/POS printer / PDF / SMS / WhatsApp
Tax Display: Inclusive / Exclusive (affects price shown)
Loyalty: Points per LKR | Expiry days | Min redemption

Config > HR Tab

Config > HR — fields:

EPF Rate Employee: 8% (editable number field)
EPF Rate Employer: 12% (editable number field)
ETF Rate: 3% (editable number field)
PAYE Tax Slabs: table with From | To | Rate %
Each row editable. + Add Slab button.
Leave Types: list — + New: Name | Paid/Unpaid
Annual days | Accrual (monthly/on join)
Bank Upload Format: select bank → downloads template

Config > Integrations Tab:

PayHere: Merchant ID | Secret Key | Sandbox on/off
Webhook URL (auto-generated): /webhook/payhere/{tenant}
PayPal: Client ID | Secret | Sandbox on/off
Webhook URL: /webhook/paypal/{tenant}
SMS — Dialog Ideamart: API Key | Sender ID
SMS — Mobitel: Username | Password | Source Address
Email: SMTP host | port | user | password OR SendGrid API Key
WhatsApp: WhatsApp Business API token | Phone number ID

Config > Email / SMS Templates Tab

Email & SMS Templates — each configurable per company:

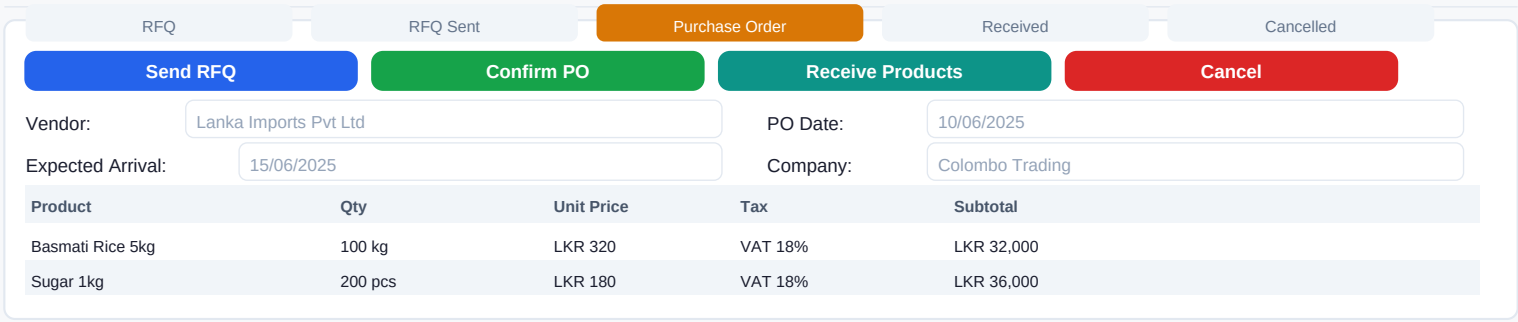
Templates list: Name | Type (email/sms) | Module | Subject | Trigger
+ New Template button: opens template editor
Template variables (auto-populated): {customer_name} {invoice_amount} {due_date} {company_name}
{order_ref} {product_list} {tracking_link} {payment_link} {cashier_name}
Built-in templates (editable): Invoice Created | Payment Received | Low Stock Alert
Subscription Renewal Reminder | GRN Confirmed | PO Approved | Payslip Ready
Preview button: renders template with sample data. Send Test button: sends to your email.
HTML editor for email templates. Plain text for SMS (160 char limit shown).

Config > Modules Tab — Enable/Disable per company:

Module toggle list: Module Name | Description | Status (On/Off) | Depends On
Manufacturing: OFF by default. Turn ON for factory clients. Adds BOM, MRP, Production menus.
Point of Sale: OFF by default. Turn ON for retail. Adds POS menu + POS Config tab.
E-Commerce: OFF by default. Turn ON to add Online Store menu.
Projects: OFF by default. Turn ON for service companies.
Field Service: OFF by default. Turn ON for maintenance/service teams.
When module turned ON: migrations run for that company (schema update if needed).
When module turned OFF: data preserved, menu hidden. Re-enable restores all data.
API: PATCH /api/config/modules body: {module: 'manufacturing', enabled: true, company_id: x}

Purchasing Module — Tabs

Purchase Order Detail Screen



Product	Qty	Unit Price	Tax	Subtotal
Basmati Rice 5kg	100 kg	LKR 320	VAT 18%	LKR 32,000
Sugar 1kg	200 pcs	LKR 180	VAT 18%	LKR 36,000

Purchasing Button Behaviours:

Send RFQ: PATCH /api/purchases/{id}/send → state=sent → email PDF to vendor → log in chatter

Confirm PO: PATCH /api/purchases/{id}/confirm → state=purchase → stock.picking created (incoming)
Validation: vendor set, at least 1 line, scheduled date set

Receive Products: Opens GRN screen (stock.picking) linked to this PO. Shows expected vs received.
After Validate GRN: qty_received updated on PO lines. If all received: PO state=done.

Create Vendor Bill (after GRN): POST /api/vendor-bills/from-po/{id} → draft account.move (type=in_invoice)
Lines pre-populated from PO. Amount editable (vendor may charge differently).

3-Way Match: System checks: PO qty == GRN qty == Bill qty (within tolerance). If mismatch: warning shown.

Landed Costs Screen (Inventory > Operations > Landed Costs):

Purpose: add import costs (freight, customs duty, insurance) to product cost price.

+ New Landed Cost button. Fields:
- Vendor Bill: link to vendor bill for freight/duty
- Transfers: select the GRN(s) this applies to
- Valuation Lines: Product | Description | Split Method | Amount
- Split Methods: Equal / By Quantity / By Current Cost / By Weight / By Volume

Validate button: allocates the cost across product lines proportionally.
stock_valuation_layer records created. product.standard_price updated.

Vendor Management Screen:

Path: Purchasing > Vendors (same as Contacts filtered to vendors)

Tabs on vendor detail: Contacts | Sales & Purchase | Accounting | Internal Notes | Purchase History

Key fields: Payment Terms | Currency | Bank Account (for payments) | Pricelist

Purchase History tab: all POs, average lead time, on-time delivery %, price trend

Vendor Rating widget (top of form): 1-5 stars, based on: delivery accuracy, price, quality (configurable weights)

Block Vendor button: sets vendor.active=false. Blocked vendors can't be selected on new POs.

Supplier Pricelist: + New Pricelist line: product | min_qty | price | validity dates

Purchasing API Endpoints:

GET	/api/purchases	List POs with filters: state, vendor_id, date, page
GET	/api/purchases/{id}	Detail with lines, GRNs, bills
POST	/api/purchases	Create PO/RFQ
PATCH	/api/purchases/{id}/send	Send RFQ email to vendor
PATCH	/api/purchases/{id}/confirm	Confirm → create GRN (stock.picking)
POST	/api/vendor-bills/from-po/{id}	Create vendor bill from PO
POST	/api/landed-costs	Create landed cost allocation
POST	/api/landed-costs/{id}/validate	Apply landed cost → update product cost prices

Manufacturing Module — Main Tabs

Products | Bill of Materials | | Work Orders | MRP | Reporting | Config

Production Order Detail Screen

Draft | Confirmed | In Progress | To Close | Done

Confirm **Mark as Done** **Scrap** **Backorder**

Product: Garment Set (12 pcs) Qty: 50 Sets

Scheduled Date: 20/06/2025 BOM: Garment Set v2

Work Orders | Misc | Analytics SO-0042

Component	To Consume	Reserved	Availability	Lot/Serial
Fabric (meters)	60 m	60 m	Available	—
Thread (spools)	12	12	Available	—

Production Order Buttons — what they trigger:

Confirm button: mrp_production.state=confirmed | Reserve stock for all components
stock_move records created for each component (type=manufacturing)
Work orders created if routing defined in BOM

Mark as Done: Triggers wizard: enter actual qty produced. If < planned: prompt Create Backorder?
stock_move.state=done for consumed components | qty_on_hand -= consumed
Finished goods: stock_move (type=manufacturing output) created and validated
mrp_production.state=done | GL entries posted: Dr FG Stock Cr WIP

Shop Floor View: tablet-optimised screen. Operator sees their work orders. Buttons:
[Start Work Order] → records actual_start_time | [Pause] → records pause reason
[Mark Done] → records actual_finish_time, qty_produced | [Report Issue] → creates quality alert

Bill of Materials (BOM) Screen:

Path: Manufacturing > Bill of Materials
Tabs: Components | Operations (routing) | By-Products | Miscellaneous
+ New BOM button: Product | Qty | BOM Type (Manufacture/Kit/Subcontracting) | Version
Components tab: + Add Component: Product | Qty | Unit | Apply on Variants
Operations tab (if MES enabled): + Add Operation: name | work_centre | duration_expected | notes
Versions: BOM can be versioned. Activate specific version. Old versions kept for reference.
MRP uses active BOM version to calculate material requirements.

MRP Run Screen:

Path: Manufacturing > MRP > Run MRP
Procurement Wizard: select Products | Date Horizon (e.g. next 30 days) | Warehouse
Run button: system calculates: demand (from SOs + forecasts) - supply (stock + POs + WOs) = shortage
MRP Proposals table: Product | Qty Needed | Qty to Make/Buy | Suggested Date | Action
Each row has buttons: Approve (create PO or Production Order) | Ignore | Merge (combine with existing)
Approve All button: creates all suggested POs and Production Orders in batch
MRP can run on schedule (e.g. every morning 07:00) via background Celery task

Analytics Dashboard — Widget Types

Dashboard Widgets — types to build:

KPI Card: single metric + target + trend arrow
e.g. 'Monthly Revenue: LKR 2.4M ↑ vs LKR 2.1M last month'
Bar Chart: sales by period / by product / by salesperson
Line Chart: trend over time (sales, stock, payroll cost)
Pie/Donut: revenue by product category / customer segment
Table widget: top customers / top products / low stock list
Gauge: target completion % (e.g. sales target 74% done)
Pivot table: cross-tab with two dimensions (product × month)

Dashboard Screens — role-based:

CEO / Owner: Revenue, Expenses, Cash, P&L summary, Top 5 customers
Sales Manager: Pipeline, Order count, Revenue vs target, Aged AR
Warehouse Mgr: Low stock, Pending GRNs, Pending DOs, Valuation
HR Manager: Headcount, Leave summary, Payroll cost, Attendance %
POS Manager: Daily sales by outlet, Best-selling products, Session stats
Finance Manager: AR, AP, Cash flow, Bank balances, VAT summary
Customise button on each dashboard: drag/drop widgets, resize, change date range

Custom Report Builder

Report Builder — how it works (Config > Reporting > New Report):

Step 1: Select Model (Sale Order / Invoice / Stock Move / Employee / etc.)
Step 2: Add Fields: drag fields from left panel to columns. Set label, format, width.
Step 3: Filters: field | operator (=, >, <, contains, between) | value | AND/OR logic
Step 4: Group By: select field to group (e.g. group by customer, or by month)
Step 5: Sort: select field and direction
Step 6: Format: Table / Chart (bar/line/pie) / Pivot / both
Save & Schedule: name the report. Set schedule: daily/weekly/monthly email to roles.
Share: make public or assign to specific roles. Export buttons: Excel | PDF | CSV on every report.

Customer Portal — Screens

Customer Portal — Login & Dashboard:

URL: https://{company}.nexaerp.lk/portal
Login: email + password (customer account created automatically when sales order confirmed for that customer)
Dashboard: My Orders | My Invoices | My Quotes | Support
Order list: SO ref | date | amount | status | Download PDF
Click order → detail with delivery tracking link
Invoice list: INV ref | due date | amount | paid/unpaid
Pay Invoice button (if unpaid) → PayHere / PayPal checkout

Customer Portal — Invoice & Payment:

Invoice detail: product list, amounts, VAT breakdown
Download PDF button → GET /api/portal/invoices/{id}/pdf
Pay Now button → POST /api/portal/payments/initiate
→ redirects to PayHere / PayPal
→ on success: webhook → invoice marked paid
→ email receipt sent automatically
Account statement: all invoices for date range. Download PDF.
Delivery tracking: link to courier or DO status page

System-Wide Development Standards

API Standards (apply to every endpoint):

Authentication: JWT Bearer token in Authorization header. Token TTL: 1 hour. Refresh token: 7 days.
Multi-tenancy: X-Company-ID header on every request. Middleware resolves tenant schema.
Response format: {success: bool, data: {}, error: null, pagination: {page, total, per_page}}
Error codes: 400 Validation | 401 Unauthenticated | 403 Forbidden/Paused | 404 Not Found | 422 Logic Error
Pagination: ?page=1&limit=50&sort=created_at&order=desc on all list endpoints
Filtering: ?filters[state]=sale&filters[customer_id]=42 on list endpoints
Webhooks: all incoming (PayHere, PayPal) verified with HMAC signature before processing

Frontend Standards (React / Next.js):

Component structure: /pages, /components/shared, /components/{module}, /hooks, /store, /api
State management: Zustand for global (user, company, permissions). React Query for server data.
Form handling: React Hook Form + Zod validation schemas. Error messages shown inline per field.
Tables: TanStack Table (virtual scroll for large datasets). Column sorting, filtering built in.
Buttons: disabled state when loading. Loading spinner replaces icon. Toast notification on success/error.
Permissions: every button/menu item checks user.permissions[module][action]. Hidden if no permission.
Dark mode: CSS variables for all colours. Toggle in top-right user menu. Preference saved per user.
Language: i18next for EN/SI/TA. All UI strings in /locales/{lang}/translation.json. RTL not needed.

Backend Standards (Python / FastAPI):

Project structure: /app/models, /app/routes, /app/services, /app/schemas, /app/core
ORM: SQLAlchemy with Alembic migrations. One migration per feature/change.
Background jobs: Celery + Redis. Queues: high (payments, webhooks) | normal (email, reports) | low (analytics)
Caching: Redis. Cache key pattern: {tenant_id}:{model}:{id} or {tenant_id}:{list}:{filters_hash}
File storage: all attachments to S3/R2. Presigned URLs for download. Max file size: 20MB.
PDF generation: WeasyPrint renders Jinja2 HTML template → PDF. Templates in /templates/{module}/
Logging: structured JSON logs. Log level per environment. All webhook events logged to webhook_log table.
Testing: pytest. Every endpoint has at least one success and one failure test. Aim for 80% coverage.